

LINUX FUNDAMENTALS FOR ODOO WORK

# Linux OS & CLI Mastery

Essential terminal commands for Odoo developers

---

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026



# Agenda

- 1 Section 1: Getting oriented in the terminal — 9 topics
- 2 Section 2: Working with files and folders — 13 topics
- 3 Section 3: Users, groups, and permissions — 7 topics
- 4 Section 4: Processes, services, logs, and editors — 10 topics
- 5 Section 5: Networking, SSH, shell setup, and scheduling — 8 topics
- 6 Section 6: Packages, resources, firewall, and Odoo operations — 9 topics

# 1 Section 1

Getting oriented in the terminal

# 1.1 pwd - Print Working Directory

- Stands for Print Working Directory and shows the full path of your current folder. [Show current path](#)
- Use it before running file commands so you know exactly where you are.
- In scripts, capture it when you need to reuse the current path.

```
pwd
```

## Print path in a message

```
echo "You are in: $(pwd)"
```

## 1.2 cd - Go to your home folder

- The cd command changes the folder your shell is working in.
- Running cd with no path takes you back to your home folder.
- The tilde character is a shortcut for your home folder.

[Return home](#)

```
cd
```

[Open home downloads](#)

```
cd ~/Downloads
```

## 1.3 cd .. - Move up one folder

- Two dots mean the parent folder of your current location. **Move up one level**
- Use `cd ..` when you need to move one level higher in the folder tree.
- Check with `pwd` after moving if you are not sure where you landed.

```
cd ..
```

### **Move up twice**

```
cd ../..
```

## 1.4 cd with absolute paths

- An absolute path starts with a slash and works from anywhere.
- Use absolute paths when you want commands to be unambiguous.
- Production commands are safer when the path is written completely.

### Open Odoo directory

```
cd /opt/odoo
```

### Open log directory

```
cd /var/log
```

## 1.5 cd with relative paths

- A relative path starts from the folder you are currently in. **Open a child folder**
- Relative paths are short and useful while working inside a project.
- They can be risky if you forget your current folder. **Move to a sibling folder**

```
cd addons
```

```
cd ../custom_addons
```



## 1.6 ls - List files

- The ls command shows files and folders in the current location.
- Use it after cd to confirm what is available before opening or editing files.
- Plain ls is quick when you only need names.

### List current folder

```
ls
```

### List a specific folder

```
ls /opt/odoo
```

## 1.7 ls -l - Long listing

- The -l option shows permissions, owner, group, size, and date.
- Use it when checking who owns a file or whether it is executable.
- Long listings are useful before changing permissions.

[Show details](#)

```
ls -l
```

[Check an Odoo config](#)

```
ls -l /etc/odoo.conf
```

## 1.8 ls -a -h - Hidden and readable sizes

- The -a option shows hidden files that start with a dot.
- The -h option makes sizes easier to read when used with -l.
- Use these flags to inspect shell files and large folders clearly.

### Show hidden files

```
ls -a
```

### Show readable details

```
ls -lah /var/log
```

## 1.9 tree - View folder structure

- The tree command displays folders and files as a visual hierarchy.
- It helps beginners understand project layouts faster than repeated ls commands.
- Limit the depth when a project has many files.

### Show current structure

```
tree
```

### Show two levels

```
tree -L 2 /opt/odoo
```

# 2 Section 2

Working with files and folders



## 2.1 mkdir - Create folders

---

- The mkdir command creates a new directory.
- Use -p when parent folders may not exist yet.
- Choose clear folder names so maintenance is easier later.

### Create one folder

```
mkdir reports
```

### Create nested folders

```
mkdir -p backups/2026/July
```

## 2.2 touch - Create empty files

- The touch command creates an empty file if it does not exist.
- If the file already exists, touch updates its timestamp.
- It is useful for quick notes, markers, and test files.

### Create a note file

```
touch notes.txt
```

### Create a log marker

```
touch /tmp/odoo-restart.marker
```

## 2.3 cp - Copy a file

- The cp command copies a file to a new path.
- Use it before editing important configuration files.
- Give backups names that include the date or reason.

### Copy a file

```
cp odoo.conf odoo.conf.bak
```

### Copy into backup folder

```
cp /etc/odoo.conf ~/backups/odoo.conf
```



## 2.4 cp -r - Copy folders

- The -r option copies directories recursively.
- Recursive copy includes files and folders inside the source folder.
- Confirm the destination before copying large project folders.

### Copy an addon folder

```
cp -r custom_addons/sale_report backups/
```

### Copy a config directory

```
cp -r /etc/nginx/sites-available ~/nginx-backup
```

## 2.5 mv - Rename files

- The mv command can rename a file in the same folder.
- Renaming does not create a second copy.
- Use clear names so old and new versions are easy to identify.

### Rename a draft

```
mv draft.txt meeting-notes.txt
```

### Rename a config backup

```
mv odoo.conf.bak odoo.conf.before-port-change
```

## 2.6 mv - Move files

- The mv command also moves files into another folder.
- Moving changes the file path but keeps the same file content.
- Check the destination folder before moving production files.

### Move a report

```
mv report.pdf archive/
```

### Move a downloaded addon

```
mv ~/Downloads/my_addon /opt/odoo/custom_addons/
```

## 2.7 rm - Remove files

- The rm command deletes files.
- Deleted files usually do not go to a recycle bin on servers.
- Use ls first when the file name contains wildcards.

### Remove one file

```
rm old-notes.txt
```

### Preview before removing logs

```
ls *.tmp
```

## 2.8 rm -r - Remove folders carefully

- The -r option deletes folders and everything inside them.
- Use recursive deletion only after confirming the exact path.
- On production systems, prefer moving a folder aside before deleting it.

### Remove a test folder

```
rm -r test-output
```

### Safer first step

```
mv old_addon old_addon.remove_after_backup
```

## 2.9 cat - Print file contents

---

- The cat command prints a whole file to the terminal.
- Use it for short files such as simple notes or small config snippets.
- Avoid cat for very large logs because it can flood the screen.

### Read a short file

```
cat README.md
```

### View Odoo config

```
cat /etc/odoo.conf
```

## 2.10 less and more - Page through files

---

- less opens a file one screen at a time.
- Use slash inside less to search within the file.
- more is simpler, but less is usually more comfortable for logs.

### Open a log

```
less /var/log/odoo/odoo.log
```

### Page through config

```
more /etc/odoo.conf
```

## 2.11 head and tail - Start or end of files

---

- head shows the beginning of a file.
- tail shows the end of a file, which is useful for recent logs.
- Use tail -f to follow new log lines while a service runs.

### Show first lines

```
head -20 /var/log/odoo/odoo.log
```

### Follow recent logs

```
tail -f /var/log/odoo/odoo.log
```



## 2.12 grep - Search text

- The grep command finds lines that contain matching text. [Search for errors](#)
- Use -i for case-insensitive searches.
- Use grep to find errors, ports, usernames, or configuration keys.

```
grep -i error /var/log/odoo/odoo.log
```

### [Find a config setting](#)

```
grep xmlrpc_port /etc/odoo.conf
```

## 2.13 find - Locate files

- The find command searches folders by name, type, size, or [Find Python files](#) time.
- Start with a specific folder so the search stays fast.
- Quote patterns that contain wildcard characters.

```
find /opt/odoo/custom_addons -name "*.py"
```

### Find log files

```
find /var/log -type f -name "*.log"
```

# 3 Section 3

Users, groups, and permissions

## 3.1 chmod numeric - Set permissions

- chmod changes read, write, and execute permissions.
- Numeric modes use 4 for read, 2 for write, and 1 for execute.
- Common modes are 644 for files and 755 for executable folders.

### Set a normal file mode

```
chmod 644 odoo.conf
```

### Make a script executable

```
chmod 755 deploy.sh
```

## 3.2 chmod symbolic - Adjust permissions

- Symbolic modes change permissions by user, group, or others.

- Use + to add a permission and - to remove one.

- Symbolic chmod is readable when making a small change.

**Add execute for owner**

```
chmod u+x backup.sh
```

**Remove write from others**

```
chmod o-w shared.conf
```

## 3.3 chown - Change owner

- chown changes which user owns a file or folder.
- The owner controls permissions marked for user access.
- Odoo files should usually be owned by the service user, not root.

### Change file owner

```
sudo chown odoo /etc/odoo.conf
```

### Change folder owner recursively

```
sudo chown -R odoo /opt/odoo/custom_addons
```

## 3.4 chgrp - Change group

- chgrp changes the group assigned to a file or folder.
- Group permissions help teams share access without using one account.
- Use groups to separate application access from administrator access.

### Change file group

```
sudo chgrp odoo /etc/odoo.conf
```

### Change addon group recursively

```
sudo chgrp -R odoo /opt/odoo/custom_addons
```

## 3.5 Users and groups overview

---

- A Linux user is an identity that owns processes and files.
- A group collects users so permissions can be shared.
- Service accounts such as odoo should run applications with limited access.

### List current user's groups

```
groups
```

### Show an account entry

```
getent passwd odoo
```



## 3.6 sudo - Run with administrator rights

- sudo runs a command with elevated privileges.
- Use it only for commands that really need administrator access.
- Read the command twice before using sudo on production servers.

### Edit a system config

```
sudo nano /etc/odoo.conf
```

### Restart a service

```
sudo systemctl restart odoo
```

## 3.7 whoami and id - Check identity

- whoami prints the username of the current shell.
- id shows the user id, group id, and group memberships.
- Check identity before changing files or running service commands.

### Show current user

```
whoami
```

### Show full identity

```
id
```

# 4 Section 4

Processes, services, logs, and editors

## 4.1 ps - List processes

- A process is a running program.
- ps shows processes and their process ids.
- Use ps to confirm whether Odoo, PostgreSQL, or Nginx is running.

### Show your processes

```
ps
```

### Find Odoo processes

```
ps aux | grep odoo
```

## 4.2 top and htop - Watch system activity

- top shows live CPU, memory, and process activity.
- htop is an easier interactive version when installed.
- Use these tools when users report slowness or high load.

[Open top](#)

```
top
```

[Open htop](#)

```
htop
```

## 4.3 kill - Stop a process by id

---

- kill sends a signal to a process id.
- The default signal asks the process to stop cleanly.
- Use service commands before kill when systemd manages the process.

### Stop a process

```
kill 12345
```

### Force stop only if needed

```
kill -9 12345
```

## 4.4 pkill - Stop processes by name

- pkill matches processes by name or pattern.
- It is faster than finding each process id manually.
- Use careful patterns so you do not stop unrelated work.

### Stop matching worker

```
pkill -f odoo-bin
```

### Preview matches first

```
pgrep -af odoo-bin
```

## 4.5 systemctl status - Check services

- systemctl manages services on systemd-based Linux systems.
- The status command shows whether a service is active and healthy.
- Check status before and after service changes.

### Check Odoo

```
systemctl status odoo
```

### Check PostgreSQL

```
systemctl status postgresql
```



## 4.6 systemctl start and stop

- start launches a service managed by systemd.
- stop asks systemd to shut the service down.
- Use these commands during planned maintenance windows.

### Start Odoo

```
sudo systemctl start odoo
```

### Stop Odoo

```
sudo systemctl stop odoo
```

## 4.7 systemctl enable - Start on boot

- enable configures a service to start automatically after reboot.
- disable prevents automatic startup without deleting the service.
- Use status to confirm both enabled state and active state.

### Enable Odoo at boot

```
sudo systemctl enable odoo
```

### Check enabled state

```
systemctl is-enabled odoo
```

## 4.8 journalctl - Read service logs

---

- journalctl reads logs collected by systemd.
- Use -u to focus on one service.
- Use -f to follow new log entries during testing.

### Read recent Odoo logs

```
journalctl -u odoo -n 50
```

### Follow Odoo logs

```
journalctl -u odoo -f
```

## 4.9 nano - Beginner-friendly editor

- nano is a simple terminal text editor.
- The most important shortcuts are shown at the bottom of the screen.
- Use nano when you need a quick safe edit and are still learning terminal editors.

### Edit a note

```
nano notes.txt
```

### Edit Odoo config

```
sudo nano /etc/odoo.conf
```

## 4.10 vim - Modal editor intro

- vim is a powerful editor with separate normal and insert modes.
- Press i to insert text, Esc to return to normal mode, and :wq to save and quit.
- Learn the basics because many servers include vi or vim by default.

### Open a file

```
vim notes.txt
```

### Save and quit

```
:wq
```

# 5 Section 5

Networking, SSH, shell setup, and scheduling

## 5.1 ip - Inspect network addresses

---

- The ip command shows network interfaces, addresses, and routes.
- Use it to confirm the server address before testing access.
- Modern Linux systems prefer ip over older ifconfig commands.

### Show addresses

```
ip addr
```

### Show default route

```
ip route
```

## 5.2 ping - Test basic reachability

- ping sends small network packets to test whether a host responds.
- It helps separate network reachability from application problems.
- Some servers block ping, so a failed ping is not always final proof.

### Ping a hostname

```
ping -c 4 example.com
```

### Ping a gateway

```
ping -c 4 192.168.1.1
```



## 5.3 ss - Check listening ports

- ss shows network sockets and listening ports.
- Use it to confirm whether Odoo is listening on its expected port.
- The -tulpn options are common for TCP, UDP, listening, process, and numeric output.

### Show listening services

```
sudo ss -tulpn
```

### Check Odoo port

```
sudo ss -tulpn | grep 8069
```

## 5.4 SSH keys - Safer server login

- SSH keys let you log in without sending a password each time.
- A private key stays on your machine and a public key goes on the server.
- Protect private keys because they prove your identity.

### Create a key

```
ssh-keygen -t ed25519 -C "odoo-admin"
```

### Copy public key

```
ssh-copy-id ubuntu@server.example.com
```

## 5.5 SSH hardening basics

- SSH hardening reduces the chance of unauthorized server access. [Edit SSH config](#)
- Common basics include key-based login, disabling root login, and limiting users.
- Always keep a working session open while testing SSH changes.

```
sudo nano /etc/ssh/sshd_config
```

### Test and reload SSH

```
sudo sshd -t && sudo systemctl reload ssh
```

## 5.6 Environment variables

- Environment variables are named values available to commands and scripts.
- They often hold settings such as paths, modes, or connection details.
- Use `printenv` to inspect them without changing anything.

### Show all variables

```
printenv
```

### Set one for this shell

```
export OD00_CONF=/etc/odoo.conf
```

## 5.7 .bashrc - Shell startup settings

---

- .bashrc runs when an interactive Bash shell starts.
- Use it for aliases, prompts, and personal environment defaults.
- Avoid putting production service secrets in a personal shell file.

### Edit bashrc

```
nano ~/.bashrc
```

### Reload bashrc

```
source ~/.bashrc
```

## 5.8 cron - Schedule recurring commands

- cron runs commands on a time schedule.
- Each crontab line includes timing fields and a command.
- Redirect output to a log file so scheduled jobs can be checked later.

### Edit user cron

```
crontab -e
```

### Nightly backup example

```
0 2 * * * /usr/local/bin/backup-odoo.sh >> /var/log/odoo-b
```

# 6 Section 6

Packages, resources, firewall, and Odoo operations

## 6.1 apt update - Refresh package lists

---

- apt update downloads the latest package index from configured repositories.
- It does not install upgrades by itself.
- Run it before installing packages on Debian or Ubuntu systems.

### Refresh package lists

```
sudo apt update
```

### Update then inspect policy

```
sudo apt update && apt policy postgresql
```



## 6.2 apt install and remove

- apt install adds packages from configured repositories.
- apt remove uninstalls packages while usually leaving config files behind.
- Read the package list before confirming installation or removal.

### Install tree

```
sudo apt install tree
```

### Remove a package

```
sudo apt remove tree
```

## 6.3 df - Check filesystem space

- df shows free and used space on mounted filesystems.
- Use -h for human-readable units.
- Check disk space when Odoo logs, backups, or attachments stop working.

### Show disk space

```
df -h
```

### Check root filesystem

```
df -h /
```

## 6.4 du - Check folder size

- du shows how much space files and folders use.
- Use -s for a summary and -h for readable sizes.
- It helps find which folder is filling a disk.

### Size of Odoo logs

```
du -sh /var/log/odoo
```

### Largest items in current folder

```
du -h --max-depth=1 | sort -h
```

## 6.5 free - Check memory

- free shows system memory and swap usage.
- Use -h to make the numbers easier to read.
- High memory pressure can slow Odoo or cause workers to restart.

### Show memory

```
free -h
```

### Watch memory changes

```
watch free -h
```

## 6.6 ufw - Basic firewall management

- ufw is a simpler firewall tool used on many Ubuntu servers.
- Allow only the ports the server actually needs.
- Check status before changing firewall rules over SSH.

### Show firewall status

```
sudo ufw status verbose
```

### Allow SSH

```
sudo ufw allow OpenSSH
```

## 6.7 Production directories for Odoo

- Common Odoo deployments place application files under [Inspect Odoo application files](#) `/opt/odoo`.
- Configuration often lives in `/etc` and logs usually live under `/var/log`.
- Know these locations before editing code or troubleshooting services.

```
ls -lah /opt/odoo
```

### Check common config path

```
ls -l /etc/odoo.conf
```

## 6.8 Odoo logs in production

- Odoo logs record startup messages, errors, warnings, and request details.
- Logs may be written to `/var/log/odoo` or collected by `journalctl`.
- Start troubleshooting by reading the newest relevant log lines.

### Follow file logs

```
tail -f /var/log/odoo/odoo.log
```

### Read service journal

```
journalctl -u odoo -n 100
```

## 6.9 Safe troubleshooting checklist

- Observe the current state before making changes.
- Change one thing at a time and record what changed.
- Verify the result with commands, logs, and user-facing behavior.
- Keep a rollback path for production fixes.

### Capture service state

```
systemctl status odoo && journalctl -u odoo -n 50
```

### Backup before editing

```
sudo cp /etc/odoo.conf /etc/odoo.conf.before-fix
```



NEXT STEP

# Try the examples on your machine

Then open the next module deck

---

Merit Advisory

01 · Linux

